

Real-Time Sydney Metro Delay Prediction

Work Report: Data Acquisition, Pipeline, Feature Engineering and Static GTFS Integration

Sachin Shrestha (S396085) and Suman Sapkota (S396224)

PRT661 Data Science Practice, Group Sydn1 Theme 1

12 September 2026

1. Purpose of this report

This report explains, step by step, the work carried out on the two roles assigned to Sachin Shrestha and Suman Sapkota in the project proposal: data acquisition and pipeline, and feature engineering and exploratory data analysis. It is written so that a reader who has not seen the code can follow what was built, why each piece was needed, what the data revealed, and what remains outstanding. Section 8 gives the exact commands needed to reproduce the result. Every figure quoted was checked directly against the project database on the date of writing.

2. The project in brief

Transport for NSW publishes live information about Sydney Metro trains through a standard called GTFS-Realtime. Three feeds are available: trip updates, which say when each train is expected at each remaining station; vehicle positions, which give each train's location, speed and how full it is; and service alerts, which describe disruptions. A separate static timetable describes what should happen on a normal day. The project collects the live feeds continuously, combines them with the timetable, and trains a model to predict how late trains will be 5, 10 and 15 minutes into the future.

Our two roles cover everything before the model: collecting the data reliably, understanding it, cleaning it, shaping it into a table a model can learn from, and integrating the static timetable. The proposal calls the result of this stage a model-ready dataset. That dataset now exists, is populated at all three prediction horizons, and is described in section 7.

3. How the work was divided

Both roles were assigned jointly. To avoid two people editing the same files, the work was split by the kind of data involved. The live feed side, meaning the exploratory analysis of the realtime feeds, the one-minute windowing, the construction of the prediction target and the data quality checks, was taken by Sachin. The timetable side, meaning downloading the static schedule, working out how it relates to the live feed, and deriving timetable-based features, was Suman's. The streaming layer and the collector deployment belong to the shared pipeline role. All of it is described here because the two halves only make sense together.

4. The process, step by step

Step 1. Look at the real data before writing any pipeline code

The first task was to call the live TfNSW feeds directly and inspect what actually comes back, rather than assume the structure from documentation. A short exploratory notebook fetched each feed, decoded it, and printed the raw records. This confirmed the field names, the id formats and the fact that times are given as whole seconds since 1970, which matters later for matching against the timetable.

Step 2. Collect a sample and study it

The ingestion pipeline was then run to store real records in a local database. A second notebook analysed everything stored: how often the feed was polled, how much data each poll returns, which columns are empty or never vary, how delay is distributed, and what the alerts contain. This notebook is deliberately self-contained, using no project code, so that the reasoning can be read on its own before it was turned into reusable modules.

Step 3. Work out what a delay reading actually is

This was the most important discovery on the live side. Each poll of the trip updates feed returns, for every running train, its expected arrival at every station it has not yet reached, up to two hours ahead. Only the stations a train has already passed carry a measured delay. The rest are the operator's own forecasts, and the furthest ones simply repeat the timetable with a delay of zero. In the first sample, of the 7,264 rows that carried an arrival time, only 2,798, or 38.5 per cent, were real measurements, averaging 10.8 seconds late; rows for stations more than 30 minutes ahead averaged just 1.4 seconds.

Averaging everything would therefore mix real measurements with a large number of meaningless zeros and make the trains look far more punctual than they are. The pipeline instead keeps, for each train in each poll, only the most recently passed station, which is the freshest genuine measurement available.

Step 4. Group the readings into one-minute snapshots

The feed is polled at slightly irregular moments and each poll covers many trains at once. To make this usable, time is cut into one-minute buckets and each bucket is summarised: the average, median, worst and best delay across active trains, how many trains were running, how many vehicles were reported, their average speed, and how many were stopped at a platform. This is the windowing described in the proposal and it produces one tidy row per minute per route.

Step 5. Decide what the model should predict

The original plan was to predict the average delay across the whole line each minute. Testing this against the collected data showed it would not work. Averaging around forty trains together cancels almost all the variation: individual delays ranged from 25 seconds early to 45 seconds late, but the line average barely moved, staying between 5.98 and 7.20 seconds. Measured statistically, the average kept only 3.6 per cent of the spread present in the per-train readings. A model asked to predict a number that never moves would appear accurate while learning nothing.

The target was therefore reframed to the individual train. Each row is one train in one minute, and the target is that same train's delay five, ten or fifteen minutes later. From the same collected data this produced 41 usable rows at the five-minute horizon instead of one. The line-wide averages are still computed, but they are now supplied as context features describing the state of the network around each train, rather than as the thing being predicted.

Step 6. Build the target without leaking the future

A prediction target must be built by looking forward in time from each row, and only forward. Because the feed was polled irregularly, taking "the next reading" could mean thirty seconds ahead in one case and fourteen minutes in another. The pipeline instead shifts a copy of the table back by exactly the horizon and joins it on the train and the minute, so a five-minute target always means five minutes on the clock. If the train was not observed at that later moment, the target is left empty rather than filled in. This guarantees that no row can see information it would not have had at the time, a mistake known as leakage that silently makes results look better than they are.

Step 7. Exclude trains that have not started

Almost half of the per-train readings, 167 of 374 in the first sample, came from trains that had not yet reached their first station. These report a delay of about zero not because they are punctual but because nothing has been measured. Including them roughly halved the apparent delay in every snapshot, pulling the window average from between 10.7 and 14.3 seconds down to between 6.0 and 7.2. The pipeline now drops them by default, with a switch to keep them if ever needed.

Step 8. Turn the notebook logic into tested program modules

Once the logic was settled in the notebook, it was moved into four modules under the processing folder. One extracts the genuine delay per train and builds the one-minute windows. One reshapes to per-train rows and attaches the targets. One adds further features: the train's own delay one, two and five minutes earlier; rolling averages and maxima over the previous five and fifteen minutes; the hour, day of week, weekend flag and peak-hour flag in Sydney local time; and the network context from the windows. One provides the data quality checks. Each module has unit tests built from small hand-worked examples where the correct answer is known in advance.

Step 9. Download the static timetable, including the calendar

The timetable is published as a single archive covering every operator in New South Wales, around 300 megabytes. The existing download code filtered it down to Sydney Metro and kept routes, trips, stops and stop times. It was extended to also keep the calendar tables, which state which version of the timetable runs on which days. This turned out to be essential for the next step. The Sydney Metro portion contains one route, 1,352 trips, 42 platforms across the stations, 28,348 scheduled stop times, and four calendar services covering Monday to Thursday, Friday, Saturday and Sunday, valid from 10 September to 9 December 2026, with one exception removing the Sunday service on 13 September.

Step 10. Find out how the live feed and the timetable relate

This was the most important discovery on the timetable side, and it did not go as the proposal assumed. When first tested, the trip ids in the live feed and the timetable never matched: not one of the 48 live trips appeared in the 1,352 timetable trips. The route ids did not match either. Only the stop ids lined up, and those matched perfectly, all 42 of them.

Inspection of the ids explained why. A metro trip id looks like 0241-001-101-016:1000. The first two segments change every time the timetable is revised, and there is one copy of each trip per day-type service. The trailing run and trip number, 101-016, identifies the physical service and survives revisions. Using that as the key matched 45 of the 48 live trips; the remaining three were runs that no longer exist in the current timetable. Every copy of a run key agrees on its direction, so the key is unambiguous about which way the train is going.

A second consequence followed. That first sample was collected on 15 August, but the timetable downloaded on 10 September is a later revision, and its scheduled times differ from the ones the live delays were measured against by about a minute on average. This means scheduled times from a mismatched static file cannot be used to recalculate delay; delay must come from the feed itself. The timetable is still valuable for what does not depend on exact times: station names and coordinates, direction and destination, how far through its journey a train is, how long the run between stations is meant to take, and the planned gap to the train ahead.

A later test settled the question completely. Data collected on 11 and 12 September, after the current timetable was downloaded, matched it exactly: every single row found its trip directly, with no fallback needed and no missing schedule columns. The mismatch was therefore never a flaw in the feeds or the method; it was purely the effect of comparing data against a timetable revised after it was collected. Collecting and refreshing on the same day removes the problem entirely.

Step 11. Build the join and the timetable features

A join module was written around these facts. It first tries an exact trip id match, for when the revisions line up. Otherwise it collects every timetable trip sharing the run key, asks the calendar which of them runs on the service day in question, and if the calendar cannot narrow it to one, prefers the copy that actually contains the stations the live train was seen at. Every row records which of these paths was taken, so downstream users know how much to trust the match. The module then attaches station names, direction, destination, position in the trip, scheduled run time from the previous station, and scheduled headway.

Headway, the planned gap to the train ahead at the same platform, exposed a trap. Because the stop times table holds a copy of the timetable for each day type, the same departure appears four times, and measuring gaps across all copies together gave a median headway of zero seconds. Measured within one day type, the figures are sensible: a minimum of 216 seconds, a median of 300, roughly four minutes on weekdays and ten at weekends, with scheduled run times between stations of 107 to 347 seconds.

Step 12. Assemble the model-ready table

A single command now runs the whole chain from the stored feeds to a finished table. It extracts genuine delays, builds the windows, reshapes to per-train rows, attaches targets, lag, rolling, temporal and network features, joins the timetable, and writes the result to a table called `model_dataset`. The modelling team reads this one table and needs nothing else.

Step 13. Add the streaming layer and the deployment file

The proposal describes the live records being published through an MQTT broker and consumed by a stream processor. This was built as an optional layer. A publisher sends every decoded record as its own message on a topic per feed, switched on with a `--publish` flag. A stream processor subscribes, validates that each record carries every required column, rejects and counts anything malformed, stores accepted records in batches, and can summarise the last few minutes into the same one-minute windows used elsewhere. The broker library is loaded only when this layer is actually used, so everything else runs without it installed.

Step 14. Fix a duplication fault found during verification

A check of the stored data against the logs revealed a discrepancy. The pipeline reported writing 7,307 alert rows per cycle, yet the alerts table held only 429 rows with no duplicates at all. The cause was that a single TfNSW alert lists one informed entity per affected stop and per affected trip, all naming the same route. Since only the route is kept, these collapsed into roughly seventeen identical rows per alert. The database write is an insert-or-update keyed on alert and route, so the duplicates were absorbed silently and the stored data was always correct, but every cycle performed seventeen times more write operations than necessary and the log message was

untruthful. Removing the duplicates at the decoding stage reduced 7,307 decoded rows to 428, with tests added so it cannot recur.

Step 15. Let the collector stop itself, then collect for an hour

Collection had never run for more than a few minutes, which was the single thing preventing the ten and fifteen minute targets from having any data. A duration option was added to the collector so a fixed-length run can be started and left alone, rather than needing to be stopped by hand, and it was covered by tests for the deadline, the minimum polling interval and clean interruption.

A one hour run was then carried out on the night of 11 September. It completed 102 cycles with no errors and stopped by itself. The effect on the dataset was decisive and is reported in the next section.

Step 16. Test everything

Sixty-nine automated tests across eight files cover the delay extraction, windowing, targets, every feature, the quality checks, the feed decoding including the duplication fix, the calendar and run key logic, the join, the headway calculation, the publisher, the stream processor and the collector's timing. They run with the standard library alone and complete in under a second. Both analysis notebooks execute from top to bottom without error, and the complete build was run end to end on real data after every change.

5. What the data showed, in plain terms

- Delays are very small. Across everything collected, no train has ever been a full minute late. The conventional 60-second "late" label therefore marks nothing, and the model must predict delay in seconds instead of sorting trains into late and on-time groups.
- Only stations already passed carry a real measurement. Everything further ahead is a forecast that sits near zero.
- Averaging all trains into one number per minute removes almost all the signal. Predicting each train separately keeps it and multiplies the usable training rows.
- Nearly half the readings are trains that have not started, which report zero and drag every average down. They are now excluded.
- The live feed and the timetable match exactly when both come from the same timetable revision. When they do not, they share station ids but not trip or route ids, and the run key plus calendar recovers the match.
- Planned headway is about four minutes on weekdays and ten at weekends, and must be measured within a single day-type timetable to be meaningful.

6. Results of the one hour collection run

The run covered 00:55 to 02:09 on Saturday 12 September, Sydney local time. It produced 837 dataset rows from 21 distinct trains, and for the first time all three prediction horizons have training data.

Measure	Before the run	After the run
Rows with a 5 minute answer	21	720
Rows with a 10 minute answer	0	631
Rows with a 15 minute answer	0	559
Timetable matched exactly	None of the August rows	All 837 rows
Rows missing schedule columns	17 per cent	Zero

The target also has genuine variation to learn from, spreading 9.3 seconds either side with values from 26 seconds early to 27 seconds late. This is the outcome the per-train reframing was designed to achieve, and it confirms that decision was correct.

An important caveat, stated plainly. The hour ran overnight, between roughly one and two in the morning Sydney time, on a Saturday. None of it is peak-hour data, and the 837 rows come from only 21 trains, tracked minute by minute, so they are far less independent than the row count suggests. No train in the entire collection has ever been more than 60 seconds off timetable, and overnight service with ten-minute gaps is precisely when delays are least likely to occur.

The pipeline is therefore proven and the modelling work can begin immediately, but the dataset is not yet broad enough to support a confident final result. What it needs is ordinary repetition: further runs covering weekday peak periods, which is when delays actually occur. Each run improves the dataset automatically, and no further engineering is required for that to happen.

7. What is in the finished dataset

The `model_dataset` table holds one row for each train in each minute it was observed, with 50 columns in five groups.

Group	What it contains
Identity	Train, route, service day, station observed, position in the trip, and how many seconds old the measurement is.
Answers to learn	That same train's delay 5, 10 and 15 minutes later, empty where the train was not seen again in time.
The train's own history	Its delay one, two and five minutes ago, the change since a minute ago, and rolling averages and worst values over the last five and fifteen minutes.
Time of day	Hour, day of week, weekend flag and peak-hour flag, all in Sydney local time.
Network and timetable context	How the whole line was running that minute, plus station name and coordinates, direction, destination, how far through the journey the train is, scheduled run time from the previous station, and the planned gap to the train ahead.

8. How to run it and get the result

All commands are run from the project folder with the virtual environment active. They are listed in the order someone reproducing the result should use them.

First time only: set up

```
python -m venv .env
source .env/bin/activate
pip install -r requirements.txt
```

A file named `.env` must exist in the project folder containing the line `TFNSW_API_KEY=your_key_here`, using a free key from the Transport for NSW Open Data portal. This file is deliberately excluded from version control so the key is never published.

Step one: download the timetable

```
python -m src.orchestration.pipeline --refresh-static
```

This downloads the New South Wales schedule, filters it to Sydney Metro and stores it. It takes a few minutes because the source archive is large. Run it on the same day as collection, so that the live feed and the timetable are the same revision and match exactly. Repeating it roughly once a day is enough.

Step two: collect live data

The collector polls every 30 seconds. Run it in the background so the terminal stays usable and the run survives closing the window. The following collects for one hour and stops by itself.

```
nohup caffeinate -i python -m src.orchestration.scheduler --minutes 60
> data/collector.log 2>&1 &
```

The trailing ampersand is what sends it to the background. On Linux, omit `caffeinate -i`; on a Mac it prevents the machine idle-sleeping mid-run, although the lid must still be left open. Progress can be watched with `tail -f data/collector.log`, and the run is finished when the log's final line reads that the scheduler stopped after a number of cycles. To check whether it is still going, or to stop it early:

```
pgrep -f src.orchestration.scheduler
pkill -f src.orchestration.scheduler
```

An hour is the practical minimum, because a fifteen minute answer needs the same train observed fifteen minutes apart. Longer runs, and runs during weekday peak hours, are what the dataset most needs now. Omitting `--minutes` makes the collector run until it is stopped, which is the setting to use if the collector is ever moved onto an always-on machine.

How much data is needed, and when to collect it

One hour proves the pipeline but does not make a defensible dataset. Two things are missing. The first is breadth of conditions: everything collected so far is off-peak, and delays are least likely to occur when the service is quiet, so a model trained only on this has never seen the situation it is meant to predict. The second is independence: the 837 rows come from just 21 trains observed minute after minute, so they carry far less information than the row count suggests. Collecting during busy periods fixes both at once, because more trains run and more of them deviate from timetable.

The timing needs care, because the team is working in Nepal and the trains run in Sydney, which is 4 hours and 15 minutes ahead. A run started in the Nepalese morning lands in the Sydney evening. The table below converts the periods worth capturing.

What it captures	Sydney time	Start in Nepal	Why it matters
------------------	-------------	----------------	----------------

Weekday evening peak	4pm to 6pm, Mon to Fri	11:45am	The most convenient run, and where delays are most likely.
Weekday morning peak	7am to 9am, Mon to Fri	2:45am	The other busy period, with different crowding patterns.
Weekday off-peak	11am to 1pm, Mon to Fri	6:45am	Gives the model a quiet weekday to contrast against.
Weekend daytime	1pm to 3pm, Sat or Sun	8:45am	Weekend timetables have wider gaps between trains.

These times hold until 4 October 2026, when Sydney moves to daylight saving and runs an hour earlier relative to Nepal. From that date subtract one hour from every start time above, so the evening peak run begins at 10:45am rather than 11:45am. Getting this wrong by an hour is easy and wastes the entire run, so it is worth confirming against a clock before starting. The dataset itself is unaffected either way, because every stored timestamp is in universal time and only converted to Sydney local time when the hour and peak features are calculated.

Use a two hour run for each, which is the same command with a larger number:

```
nohup caffeinate -i python -m src.orchestration.scheduler --minutes 120
> data/collector.log 2>&1 &
```

Four such runs would take the dataset from one off-peak hour and 21 trains to roughly eight hours across four distinct conditions, with several hundred distinct trains. That is enough to train and honestly validate a model, because rolling-origin validation needs several separate periods to test against rather than one continuous block. The runs do not have to happen on the same day, and every one of them adds to the dataset automatically, since the build step always reconstructs the table from everything stored. A single always-on machine collecting continuously would of course supersede all of this, and remains the better long-term answer.

Step three: build the dataset

```
python -m src.orchestration.pipeline --build-dataset
```

This reads everything stored, applies every step described in this report, and writes the `model_dataset` table. It takes a second or two and can be re-run at any time; it always rebuilds from scratch, so it can never drift out of date.

Step four: check the result

```
python -c "
from src.storage.db import get_connection, read_table
```

```
ds = read_table(get_connection(), 'model_dataset')
print('rows:', len(ds), 'trains:', ds.trip_id.nunique())
print('targets:', {c: int(ds[c].notna().sum()) for c in ds if c.startswith('target_')})
print('match:', ds.match_method.value_counts().to_dict())
"
```

All three target counts should be greater than zero. If the ten and fifteen minute counts are zero, collection did not run long enough or was interrupted. If the match summary shows anything other than exact matches, the timetable is from a different revision than the live data, which is corrected by re-running step one.

Running the tests

```
python -m unittest discover -s tests -t .
```

All sixty-nine tests should pass in about one second. They require no network connection, no database and no message broker.

Optional: the streaming layer

This is only needed to demonstrate the message broker described in the proposal. With a broker such as Mosquitto running, add `--publish` to the collector to send each record to it, and start the consumer in a second terminal:

```
python -m src.streaming.processor
```

9. Where everything lives

Location	What it is
notebooks/data_ingestion_and_processing.ipynb	First look at the raw live feeds, before any pipeline code.
notebooks/realtime_eda.ipynb	Analysis of the collected live data, where the delay, windowing and target decisions were worked out.
notebooks/static_gtfs_eda.ipynb	Analysis of the timetable and of how it relates to the live feed, where the join design was worked out.
src/processing/windowing.py	Extracts the genuine delay per train and builds one-minute windows.
src/processing/targets.py	Reshapes to per-train rows and attaches the 5, 10 and 15 minute targets.
src/processing/features.py	Lag, rolling, time-of-day and network context features.
src/processing/quality.py	Poll cadence, missing values and measurement-versus-forecast checks.
src/processing/join_static.py	Matches live trains to the timetable and attaches timetable features.
src/processing/decode_feeds.py	Decodes the feeds; extended to keep the calendar tables and to remove duplicate alert rows.
src/orchestration/pipeline.py	Entry point: collect once, refresh the timetable, or build the dataset.
src/orchestration/scheduler.py	Runs collection on a fixed interval, optionally for a fixed duration.
src/streaming/	MQTT publisher and stream processor.
src/storage/db.py	Reads and writes the database, including the model-ready table.
tests/	Sixty-nine automated tests across eight files.

10. Closing

The work scheduled for weeks three and four of the proposal is complete. The machinery for producing the model-ready dataset is built, tested, and has now been demonstrated end to end on live data: an hour of collection produced training rows at all three prediction horizons, matched perfectly against the timetable, with real variation in the values to be predicted.

Two findings matter most for the rest of the team. The prediction target must be the individual train measured in seconds, because a line-wide average cancels the signal and a late-or-not label cannot separate anything. And the timetable must be refreshed alongside collection, because a matching revision joins perfectly while a stale one does not join at all.

Nothing in the pipeline now blocks the modelling stage, which can begin immediately. The one thing the project still needs is ordinary repetition of the collection run, especially across weekday peak periods, and that requires no further engineering.